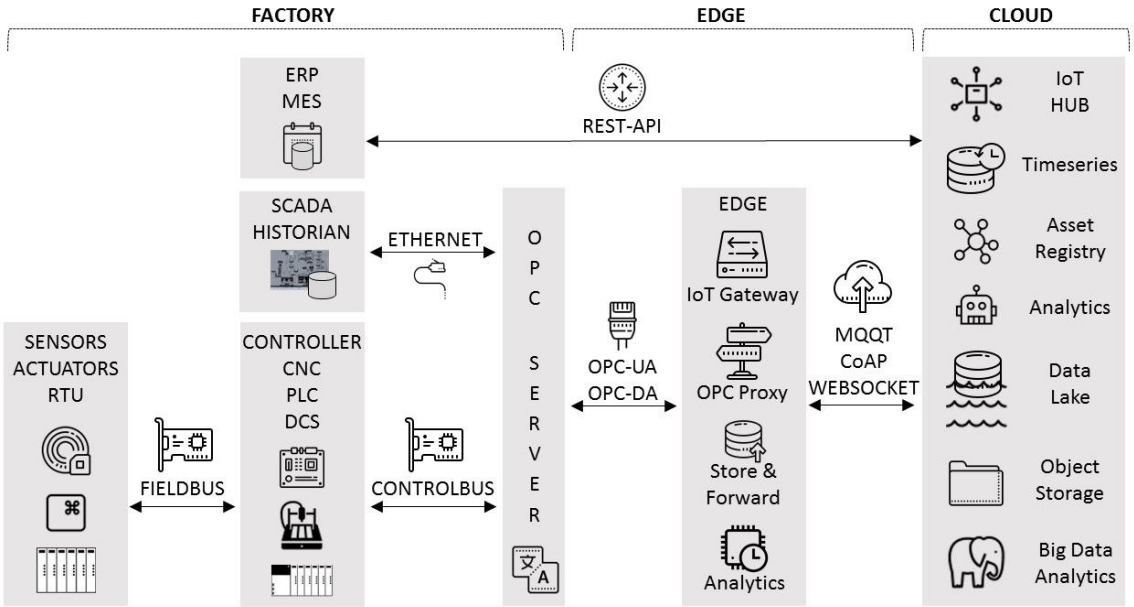
El flujo de datos en I-IoT

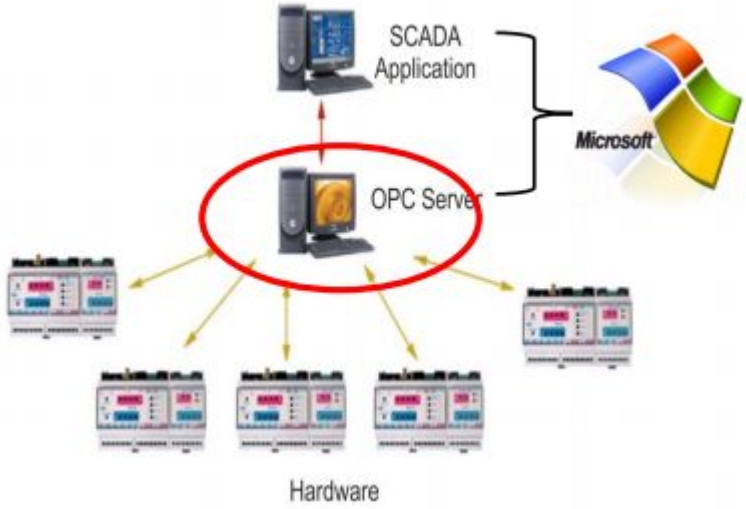
flujo de datos industriales desde los sensores hasta cuando se procesan en la nube



OPC y OPC-UA

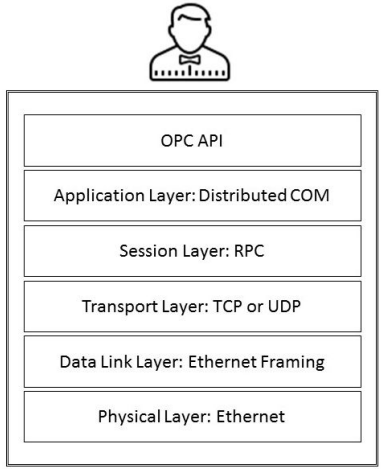
Introducción

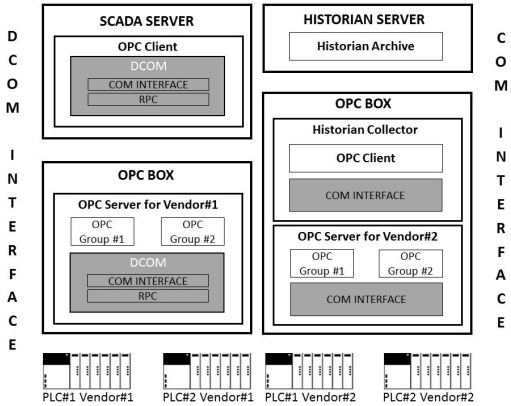
- Interfaz utilizada para cx con diferentes dispositivos y aplicaciones industriales, independientemente del fabricante, software o protocolos.
- Análogo a un traductor de idiomas
- 1996. Inicialmente, basado en el SO Windows y la tecnología Object Linking and Embedding (OLE) para control de procesos (OPC). Estas especificaciones ahora se conocen como OPC Clásico
- 2006, OPC - UA (Independiente del SO)



OPC Clásico y sus capas

- COM (Component Object Mode) es una arquitectura de software desarrollada por Microsoft para crear aplicaciones basadas en componentes
- Los DCOM (Distributed COM) son versiones de COM compatibles con la red
- Los DCOM utilizan el mecanismo de las RPC (llamada a procedimiento remoto) para enviar y recibir información entre componentes COM de forma transparente en la misma re





El modelo de datos en OPC Clásico

- Objetivo:
  - Estructurar los datos en el lado del servidor para simplificar la recopilación de datos en el lado del cliente.
  - Definir los servicios de comunicación y los mecanismos estándar para el intercambio de datos.
- Mecanismo:
  - El servidor gestiona todos los datos disponibles.
  - El servidor envía consultas sobre los datos provenientes de los dispositivos a pedido y actualiza una caché interna de forma regular. El servidor inicializa y administra una caché para cada grupo de variables solicitadas por un cliente OPC.
  - Cada pieza de datos intercambiados tiene marca de tiempo y marca de calidad.
  - El intercambio de datos incluye leer, escribir y actualización automatica. La lectura, la realiza el cliente OPC, que envía consultas sobre los datos de un grupo de forma periódica. La etapa de escritura puede ser sincrónica o asincrónica. Las actualizaciones automáticas, utilizan una tasa de actualización de solicitudes proporcionada por el cliente OPC.

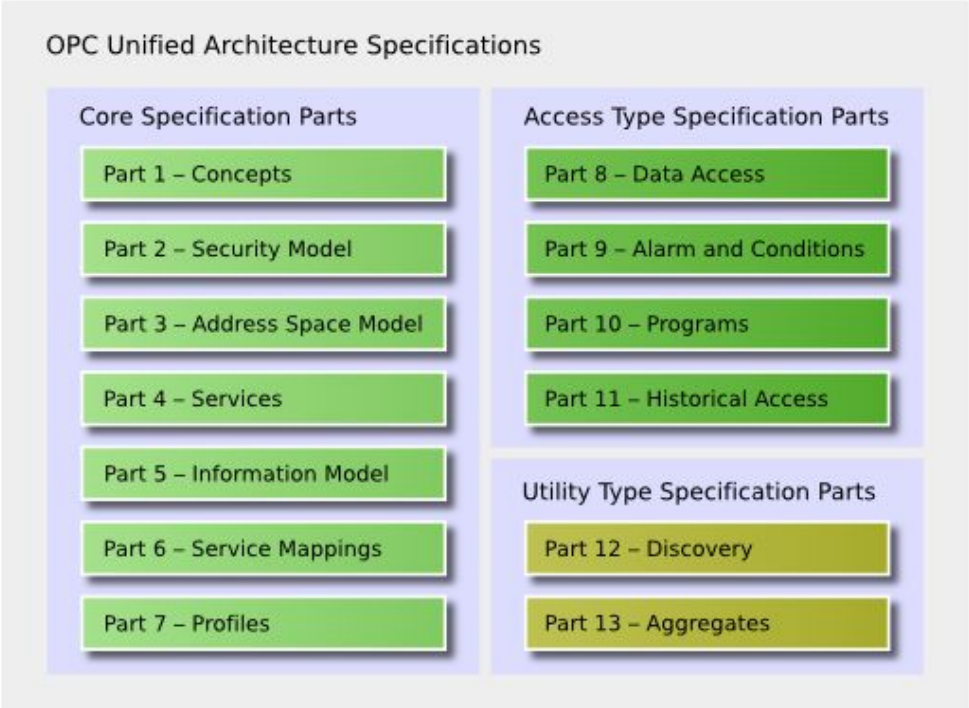
Limitaciones OPC Clásico

- El estándar se acopló a una tecnología específica. OPC Classic, de hecho, se construyó alrededor y sobre la tecnología de Microsoft.
- El tráfico COM depende de que se abran varios puertos de red indeterminados, por lo que a menudo los firewalls lo bloquean.
- Los DCOM y los RPC eran mecanismos pesados y complicados. Los objetos DCOM a menudo adolecían de falta de rendimiento y eran difíciles de mantener

OPC-UA

- Se desarrolló con el objetivo de reemplazar todas las versiones existentes basadas en COM y superar los problemas de seguridad y rendimiento, satisfaciendo la necesidad de interfaces independientes de la plataforma y permitiendo la creación de modelos de datos ricos y extensibles para describir sistemas complejos sin perder funcionalidad.
- Orientado a servicios (SOA)
- Objetivos:
  - utilizar componentes OPC en plataformas que no son Windows
  - posibilitar su integración en dispositivos pequeños
  - Implementar comunicación estándar a través de sistemas basados en firewall.

IEC 62541



IEC TR 62541-1:2016
OPC unified architecture - Part 1: Overview and concepts

IEC TR 62541-2:2016
OPC unified architecture - Part 2: Security Model

IEC 62541-3:2020 RLV
OPC Unified Architecture - Part 3: Address Space Model

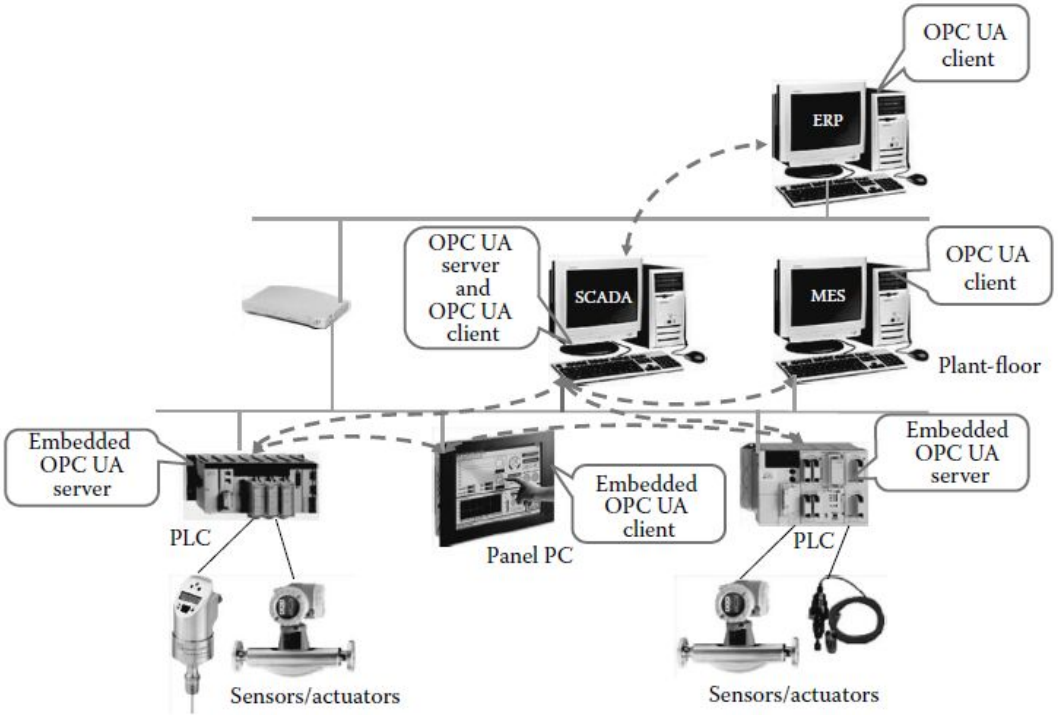
IEC 62541-4:2020 RLV
OPC Unified Architecture - Part 4: Services

IEC 62541-5:2020 RLV
OPC Unified Architecture - Part 5: Information Model

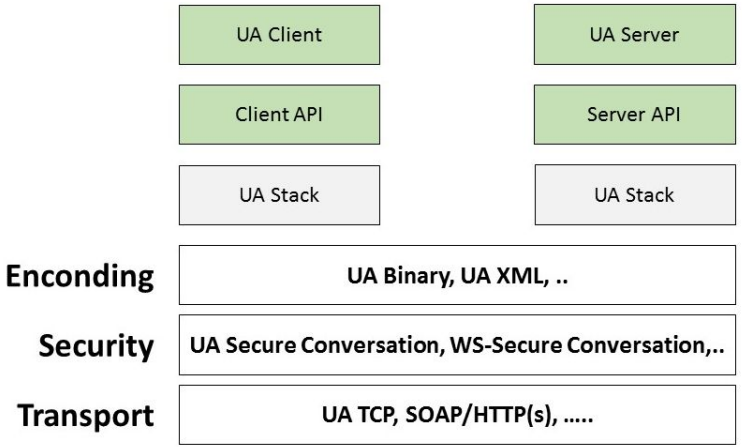
IEC 62541-6:2020 RLV
OPC Unified Architecture - Part 6: Mappings

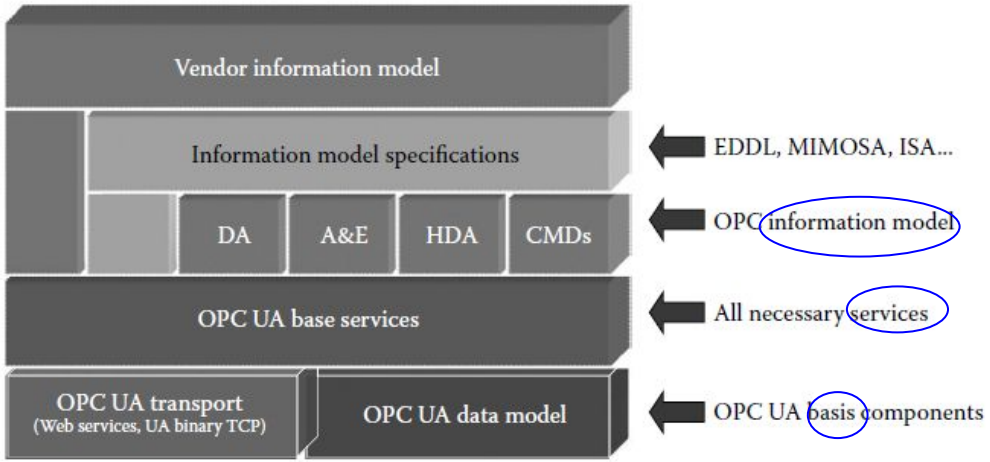
IEC 62541-7:2020 RLV
OPC unified architecture - Part 7: Profiles

Ejemplo de aplicación OPC UA



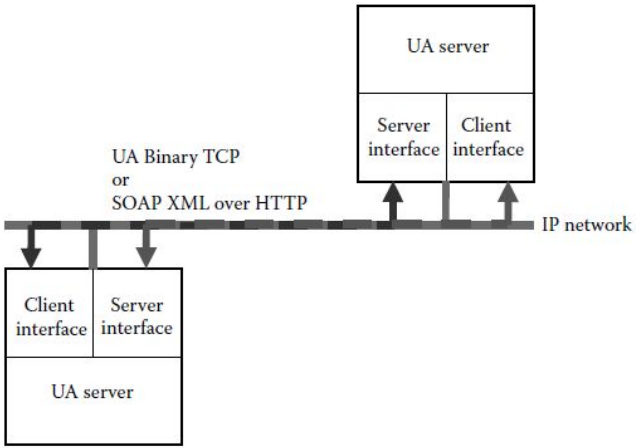
Pila y Arquitectura de software

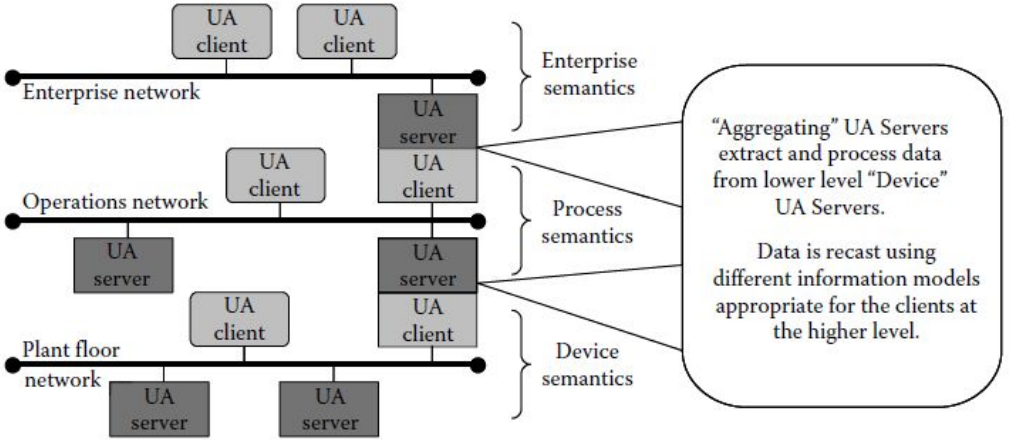




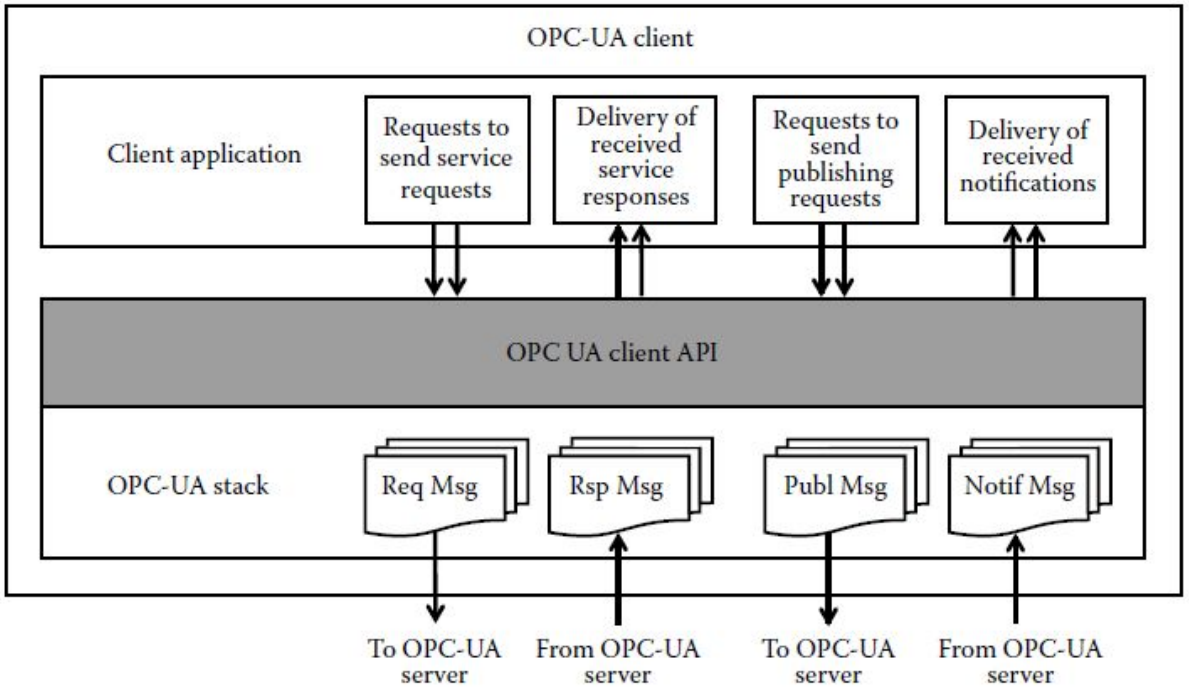
- La API aísla el código del cliente y del servidor de la pila OPC UA
- La pila UA convierte las llamadas a la API en mensajes
  I

Interacción cliente - Servidor

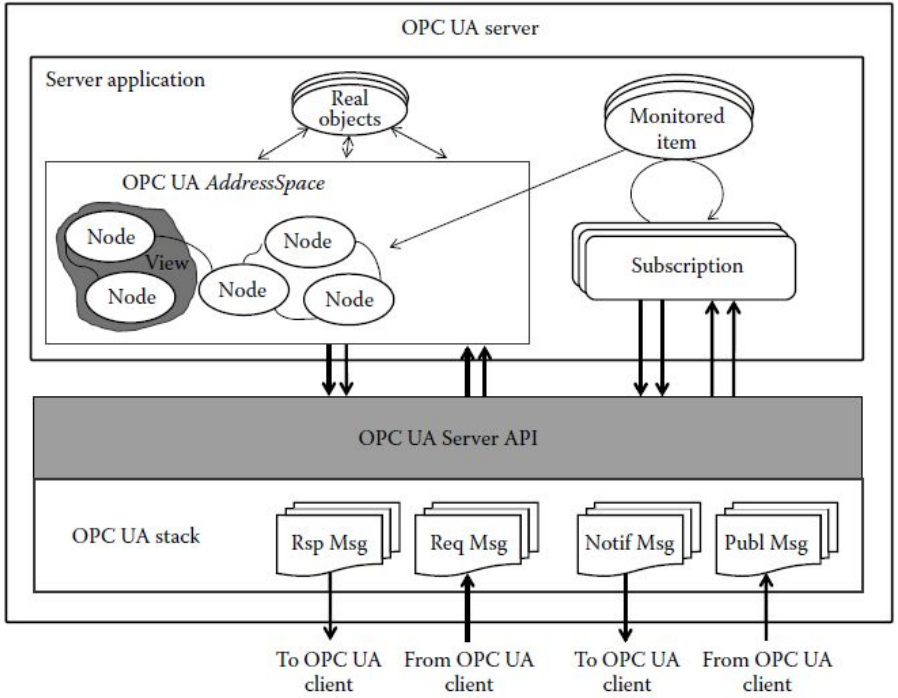




Cliente OPC-UA



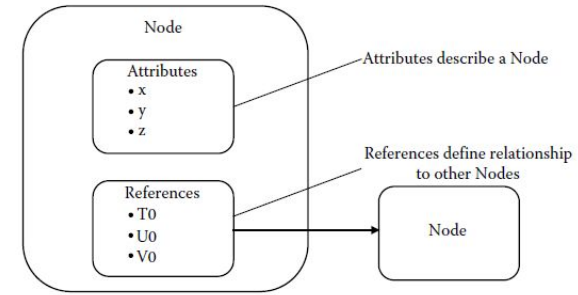
Servidor OPC-UA



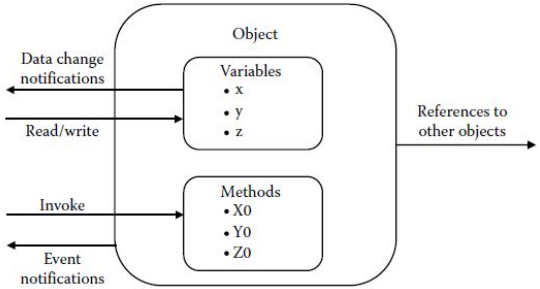
Modelado de la Información: Espacio de direcciones

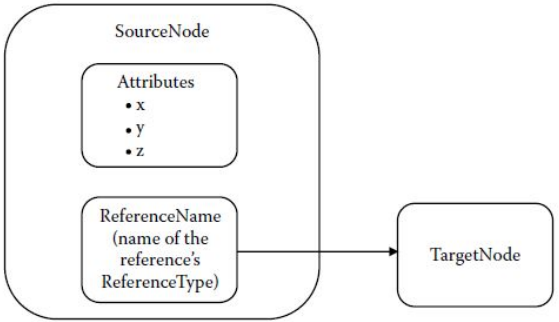
- El **modelo de información** es la representación conceptual de los **objetos** que gestiona.
- Los objetos se definen en términos de variables y métodos
- El conjunto de objetos e información relacionada que un servidor OPC UA pone a disposición de los clientes es el **espacio de direcciones**
- El espacio de direcciones del OPC UA es un conjunto de **nodos** (elementos del modelo) que están conectados por **referencias**. Cada nodo tiene propiedades, que se denominan **atributos.**

Modelado de la Información: Espacio de direcciones



UA AddressSpace node model.





UA reference model.

- Cada Nodo se asigna a una NodeClass y cada NodeClass representa un elemento diferente del modelo de objetos.
- Los clientes pueden acceder a los valores del Atributo usando servicios Leer,escribir, consulta, suscripción
- Las referencias se utilizan para relacionar los nodos entre sí. Se puede acceder a una referencia mediante los servicios de browser y query.
- El nodo que contiene la referencia se denomina SourceNode y el nodo al que se hace referencia se denomina TargetNode.

Modelos de Información estandarizados

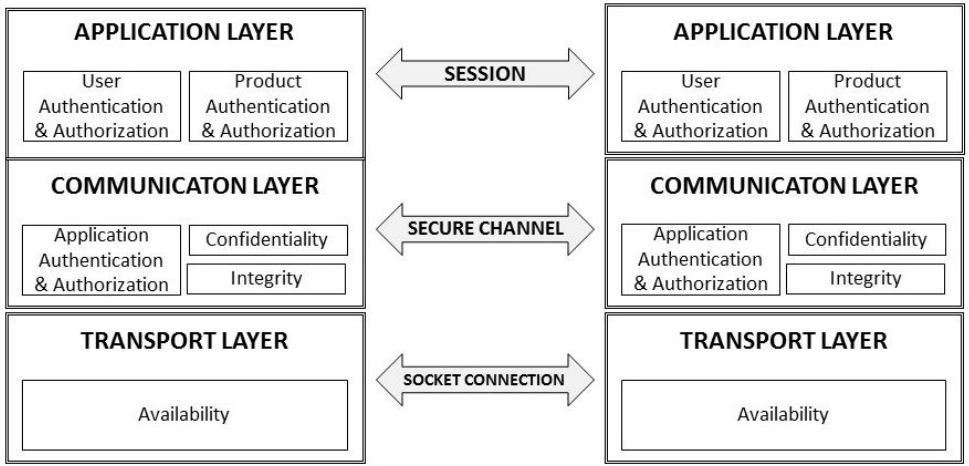
- OPC UA Data Access (Part 8)
- OPC UA Alarm and Conditions (Part 9)
- OPC UA Programs (Part 10)
- OPC UA Historical Access (Part 11)

Servicios UA

- ❏ General Services
- ❏ Discovery Service Set
- ❏ SecureChannel Service Set
- ❏ Session Service Set
- ❏ NodeManagement Service Set
- ❏ View Service Set
- ❏ Query Service Set
- ❏ Attribute Service Set
- ❏ Method Service Set
- ❏ MonitoredItem Service Set
- ❏ Subscription Service Set

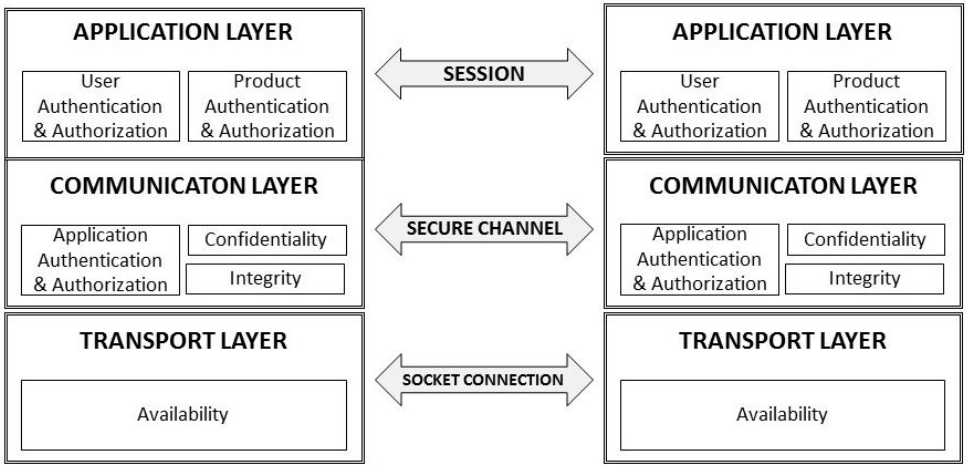
Sesión

OPC UA proporciona una comunicación cliente-servidormodelo que incluye información de estado. Esta información de estado está asociada a una sesión .



Seguridad

- Implementada mediante la definición de un **canal seguro**, en el que se basa una **sesión**.
- Un canal seguro hace que el intercambio de datos sea seguro de diferentes formas:
  - Asegura la integridad de los datos usando firmas digitales.
  - Garantiza la confidencialidad mediante cifrado.
  - Realiza la autenticación y autorización de aplicaciones mediante certificados X.509



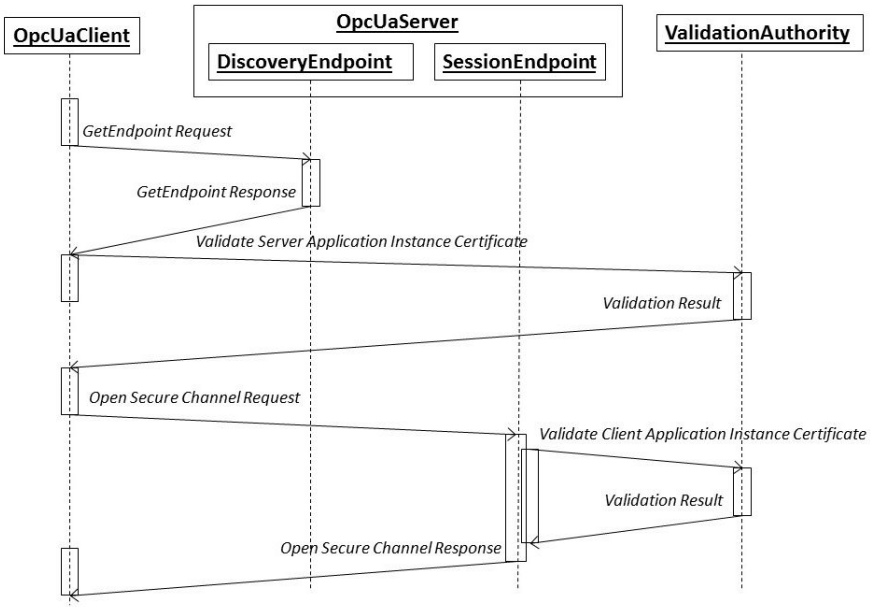
- La creación de un canal seguro se basa en un **endpoint** que ofrece cada servidor. Cada endpoint tiene las siguientes características:
  - URL del punto final
  - Certificado.
  - Política de seguridad
  - Modo de seguridad
  - Autenticación
  - Protocolo de transporte

Seguridad

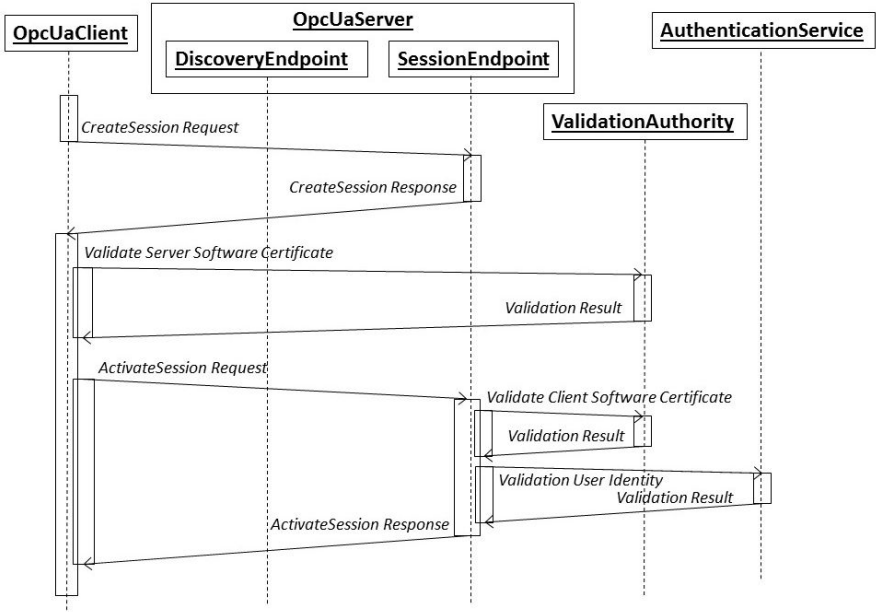
- La creación de un canal seguro se basa en tres **servicios de descubrimiento**:
  - FindServers ,
  - RegisterServer
  - GetEndpoints .
- También se tiene un servidor de descubrimiento, donde están registrados todos los servidores OPC UA activos (RegisterServer).
- Es responsabilidad del cliente solicitar al servidor de descubrimiento una lista de los servidores OPC UA activos (usando FindServers) y luego solicitar al servidor elegido y la lista de puntos finales disponibles (usando GetEndpoints).

Seguridad: Sec. de cx

- Para establecer una conexión entre un cliente OPC UA y un servidor OPC UA, tenemos que seguir 4 pasos.



Pasos 1 y 2



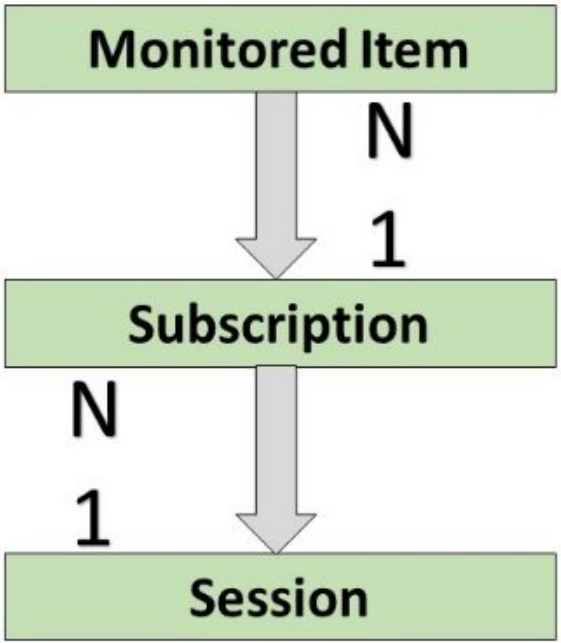
Pasos 3 y 4

Intercambio de datos

- Servicios de lectura y escritura: están optimizados para transferir una gran cantidad de datos en lugar de una sola pieza de datos o unos pocos valores.
- Parametros del servicio de lectura:
  - maxAge : este es el tiempo máximo que se tarda en devolver los valores. Lo especifica el cliente. SimaxAgese establece en cero, el servidor debe proporcionar el valor actual leyéndolo siempre directamente desde el dispositivo.
  - Tipo de marcas de tiempo : En OPC UA, se definen dos marcas de tiempo, la marca de tiempo de origen y la marca de tiempo del servidor. La marca de tiempo de origen es la marca de tiempo que proviene del dispositivo, mientras que la marca de tiempo del servidor es la marca de tiempo que proviene del sistema operativo donde se ejecuta el servidor OPC UA.
  - Lista de nodos y atributos :
    - NodeId
    - AttributeId por valor de instancia
    - Codificación de datos : esto permite al cliente elegir la codificación de datos adecuada y los valores predeterminados son XML , UA binario

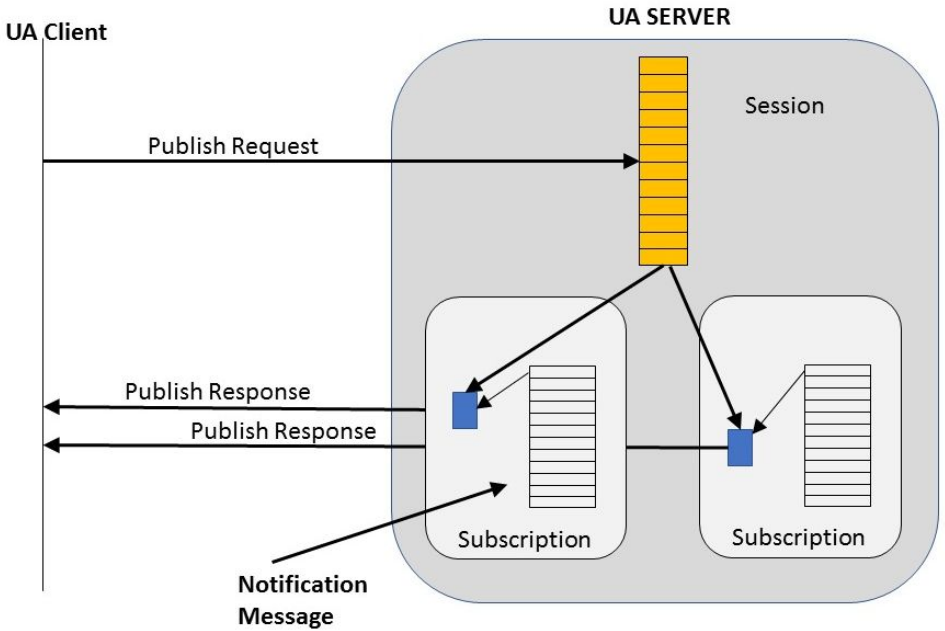
Suscripciones OPC UA

- Una suscripción nos permite crear un mecanismo automático para escribir mensajes que se enviarán al cliente, llamado NotificationMessage.
- Parmetros:
  - PublishingInterval : Esto define el intervalo de tiempo utilizado por el servidor para crear unNotificationMessagepara la suscripción dada.
  - PublishingEnabled : este es un valor booleano que habilita el reenvío delNotificationMessageal cliente.
- Los elementos supervisados se definen dentro de una suscripción ypueden ser:
  - Atributos
  - Variables
  - Objeto y vista



Notificaciones

- El cliente puede definir una o más suscripciones dentro de una sesió activa. Para cada suscripción, puede crear elementos supervisados, que producen notificaciones que se almacenan en colas específicas.
- Para cada solicitud de publicació recibida, el servidor selecciona un suscripción, elige una NotificationMessagey se le envía una respuesta de publicación



OPC UA

Llave	Valor
Fuente abierta	si
La capa OSI	Transporte o aplicación
Tipos de datos	Entero, flotante, cadena, booleano, fecha, hora, etc.
Limitaciones	No apto para una arquitectura compleja
Posibles operaciones	Leer / escribir / monitorear / consultar variables
Latencia	Bajo
Uso	IIoT
Seguridad	si
Compresión	si

Práctica

Intro

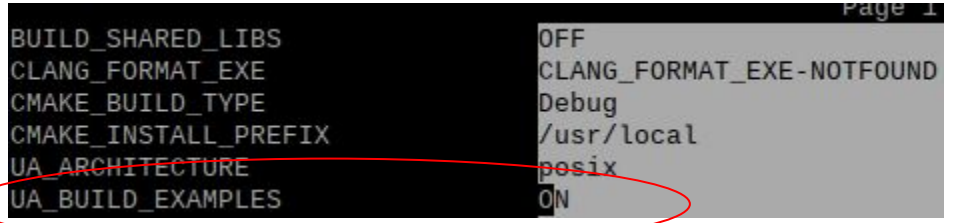
- Simularemos un flujo de datos industrial simple para implementar el intercambio de datos mediante OPC UA
- Materiales:
  - Nodejs
  - Node-RED
  - RPI
  - libreria open62541
  - opcua-client

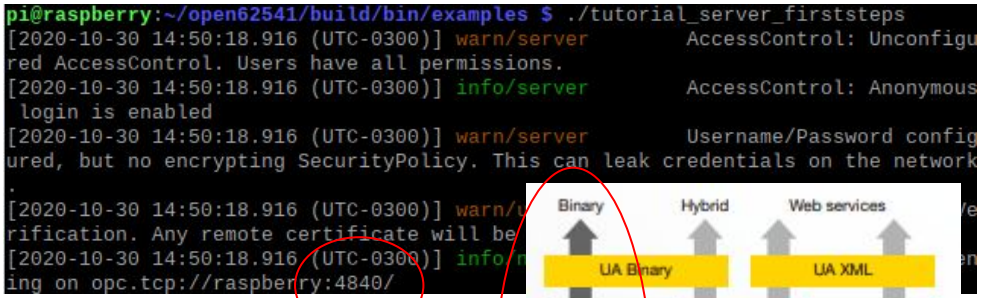
Servidor OPC-UA en RPI

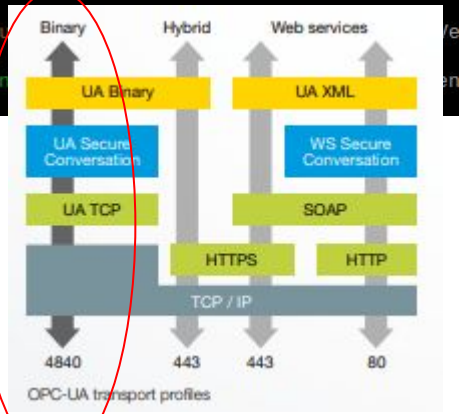
- Usando open62541 (https://open62541.org/ - https://open62541.org/doc/current/index.html)
  - Caracteristicas soportadas: https://github.com/open62541/open62541/blob/master/FEATURES.md
- Instalación de dependencias:
  - sudo apt-get install git cmake **cmake-curses-gui** build-essential gcc python3-pip qttools5-dev python3-pyqt5 libmbedtls-dev check libsubunit-dev python-sphinx graphviz python-sphinx-rtd-theme
- Creación de Servidor
  - Clonar
    - git clone https://github.com/open62541/open62541.git
  - Compilar
    - cd open62541
    - mkdir build
    - cd build
    - cmake ..
    - **ccmake ..**

Servidor OPC-UA en RPI

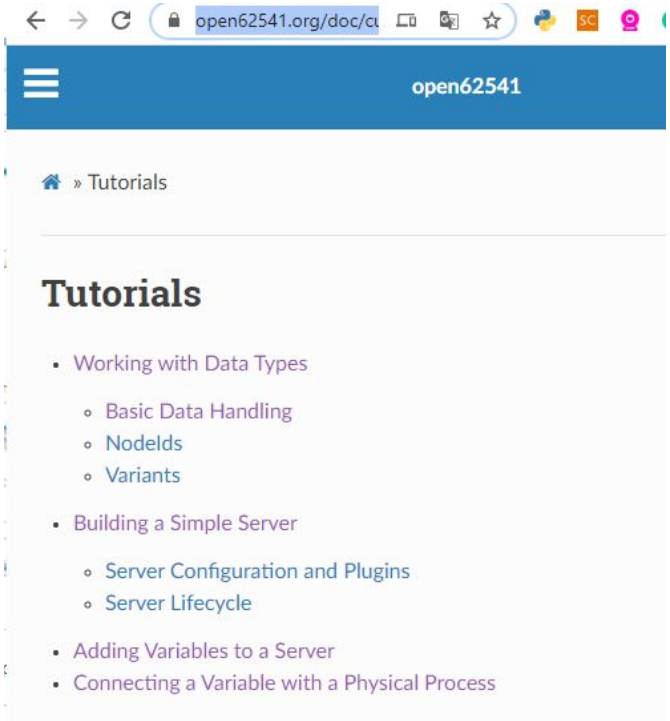
- En ccmake: Activar la opción **UA_BUILD_EXAMPLES**,
- Luego configure (c) y genere (g) el archivo make
- Compilar
  - sudo make install
- Si hacemos:
  - cd bin/examples
  - ./tutorial_server_firststeps
- ...El servidor se estará ejecutando en la RPI en el puerto 4840





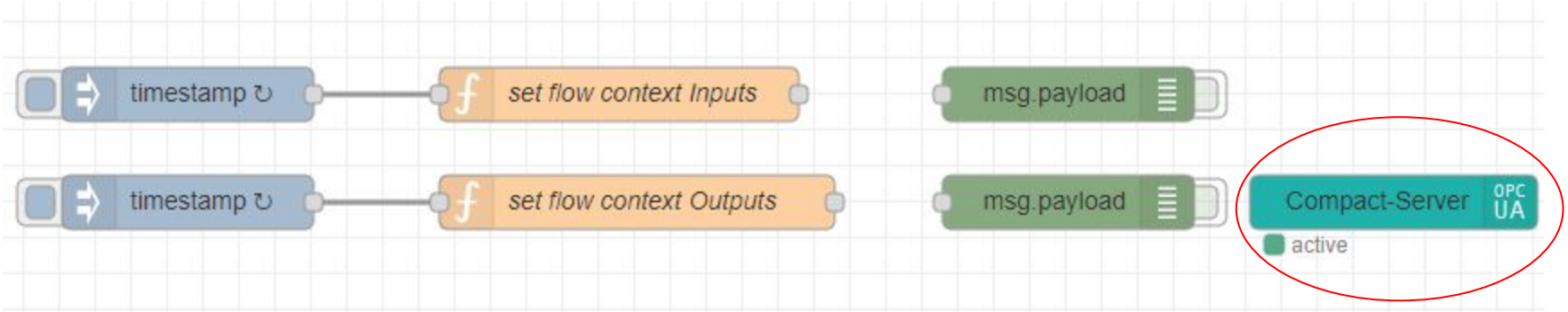


Pueden continuar con...

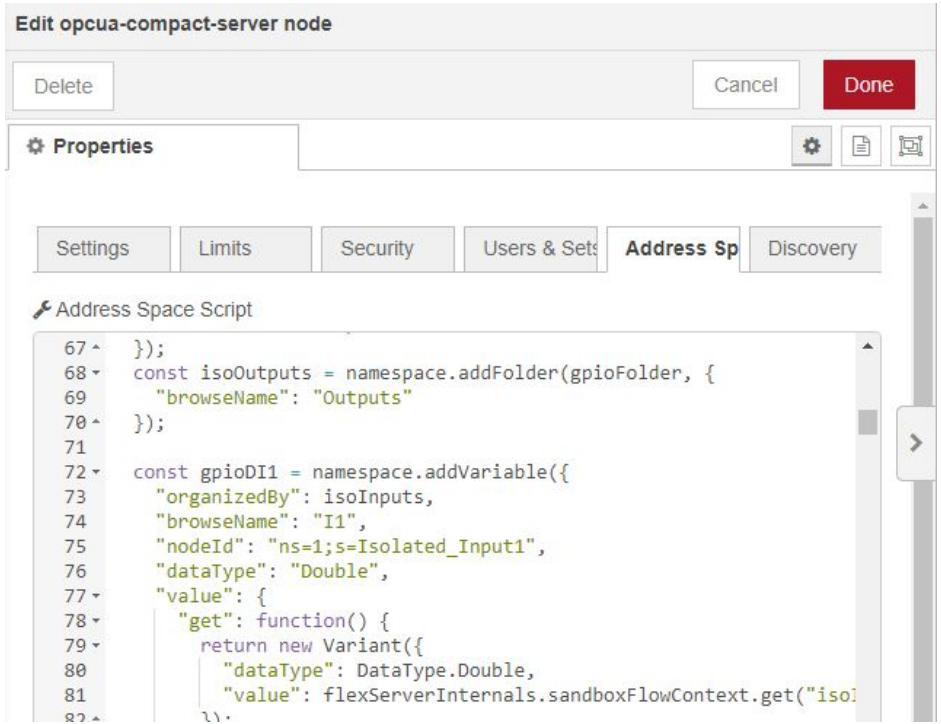


Servidor OPC UA con node-red

https://github.com/BiancoRoyal/node-red-contrib-opcua-server



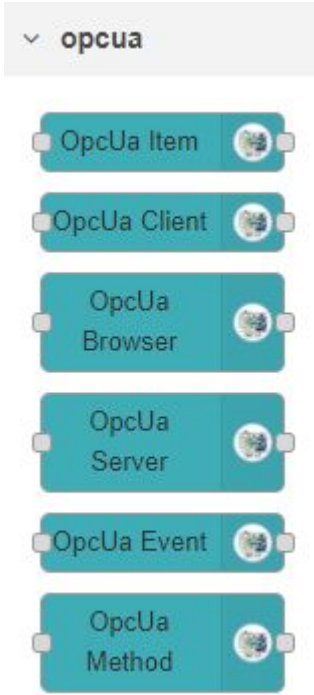
Servidor OPC UA con node-red



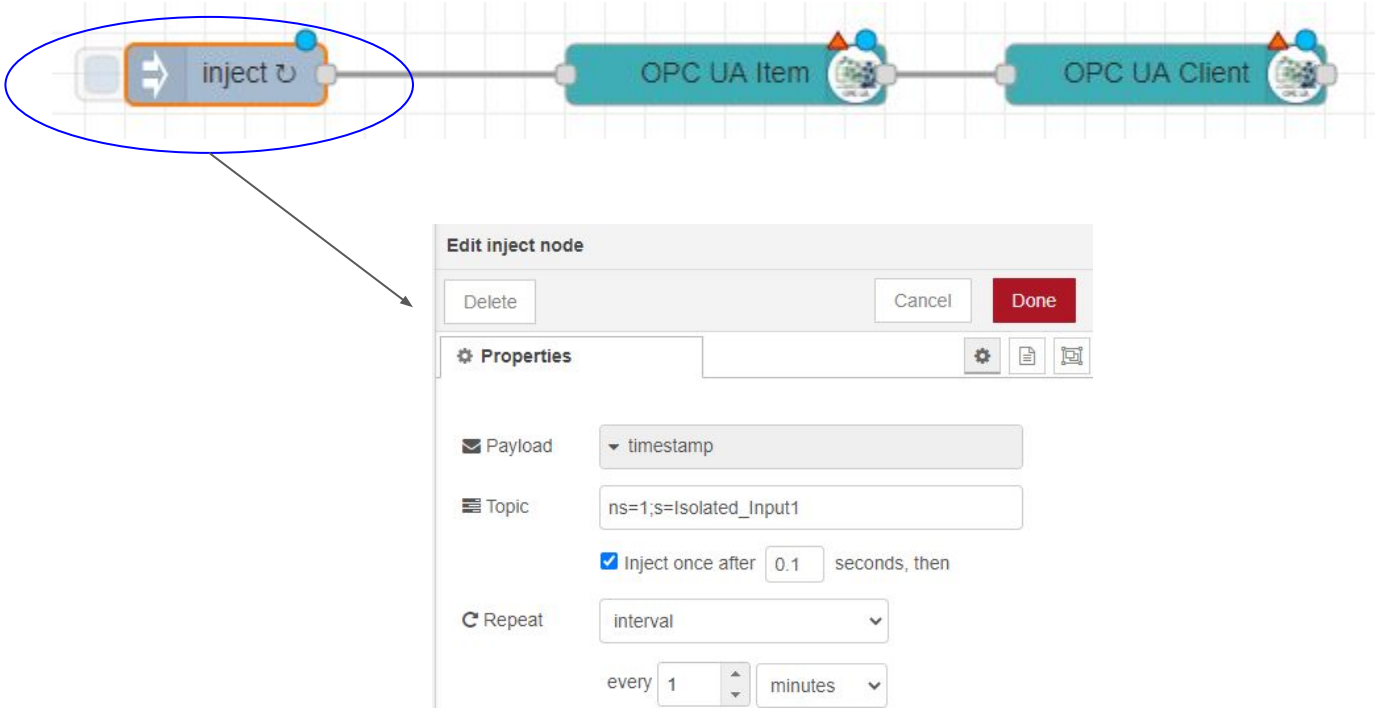
Cliente OPC UA con Node-Red

- CMD >> C:\Users\natii>node-red
- Browser >> https://localhost:1880
- Menu >> setting >> Palette >> Install >> node-red-contrib-opcua



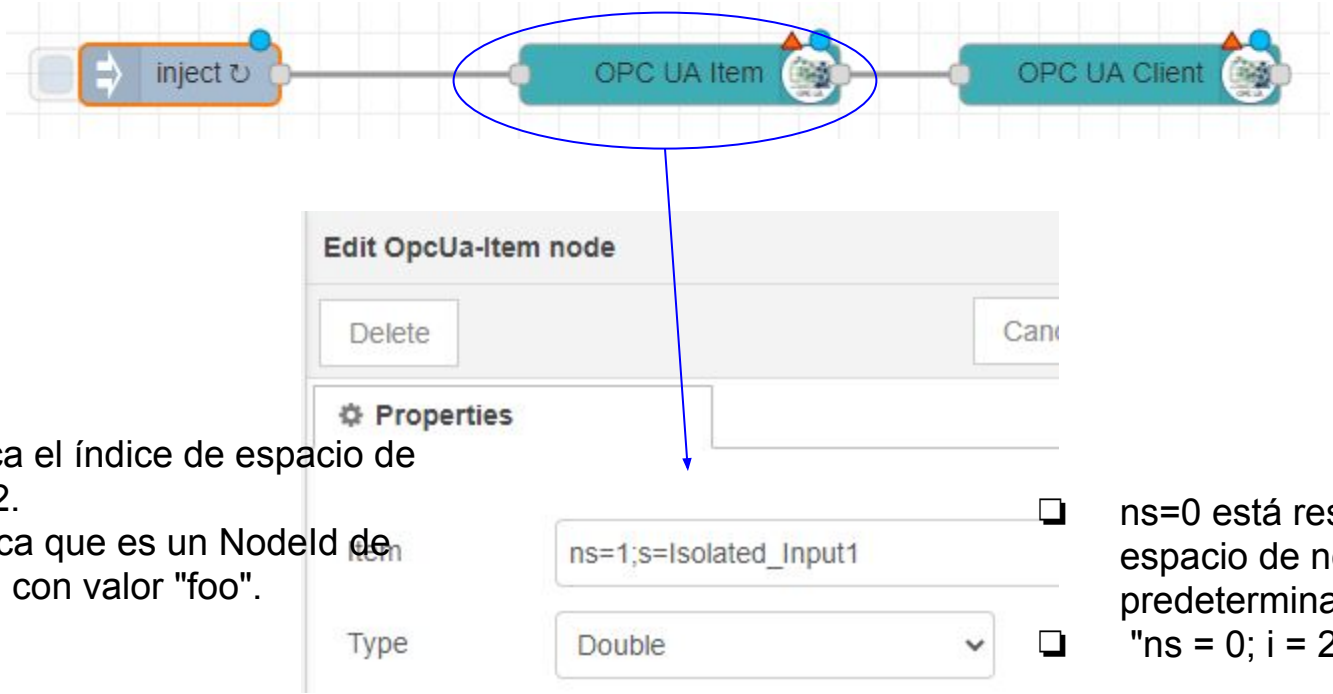


Cliente OPC UA con Node-Red



Cliente OPC UA con Node-Red

Indicar que medida obtener

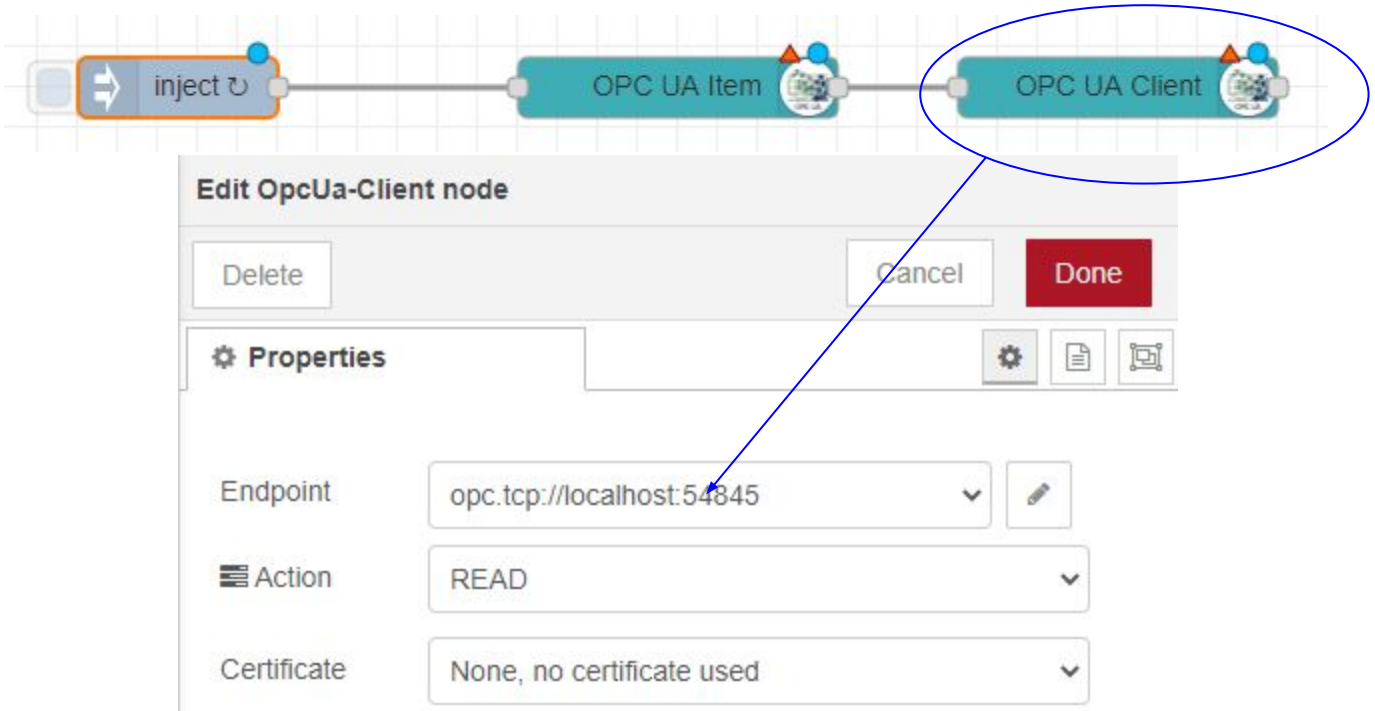


- ns=2 indica el índice de espacio de nombres 2.
- s=foo indica que es un NodeId de tipo String con valor "foo".
-

- ns=0 está reservado para el espacio de nombres predeterminado de OPC UA "ns = 0; i = 2255"
-

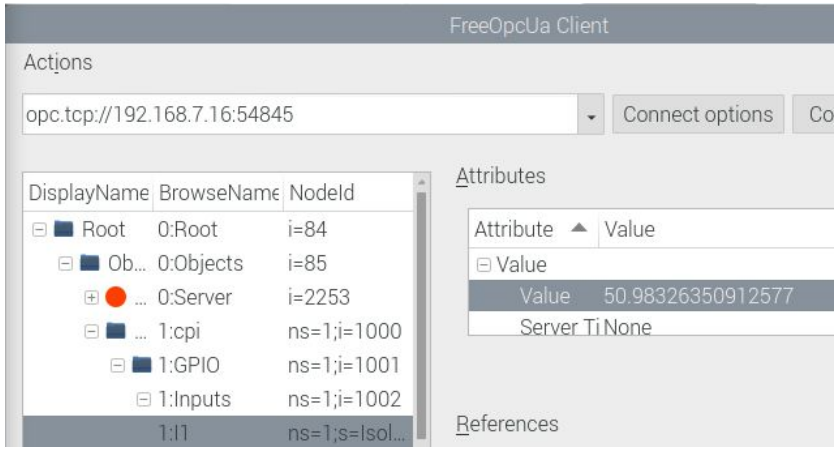
Cliente OPC UA con Node-Red

opc.tcp://192.168.7.20:4840 o localhost:54845

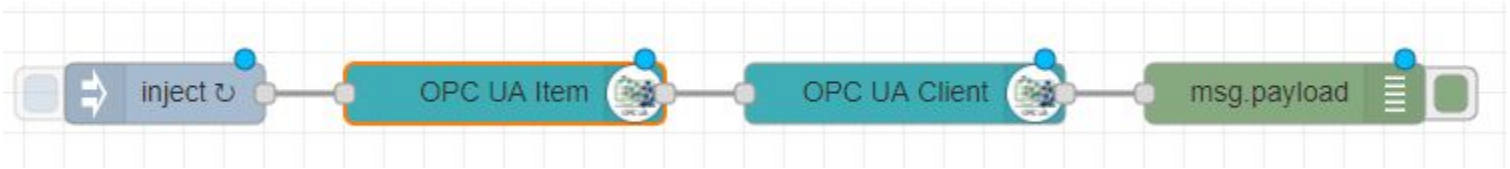


Cliente OPC UA en Linux

- https://github.com/FreeOpcUa/opcua-client-gui
- Instalación de dependencias:
  - sudo apt-get install git cmake cmake-curses-gui build-essential gcc python3-pip qttools5-dev python3-pyqt5 libmbedtls-dev check libsubunit-dev python-sphinx graphviz python-sphinx-rtd-theme python3-numpy
  - sudo pip install pyqtgraph
- Instalación cliente
  - sudo pip3 install opcua-client
- Iniciar el cliente:
  - opcua-client



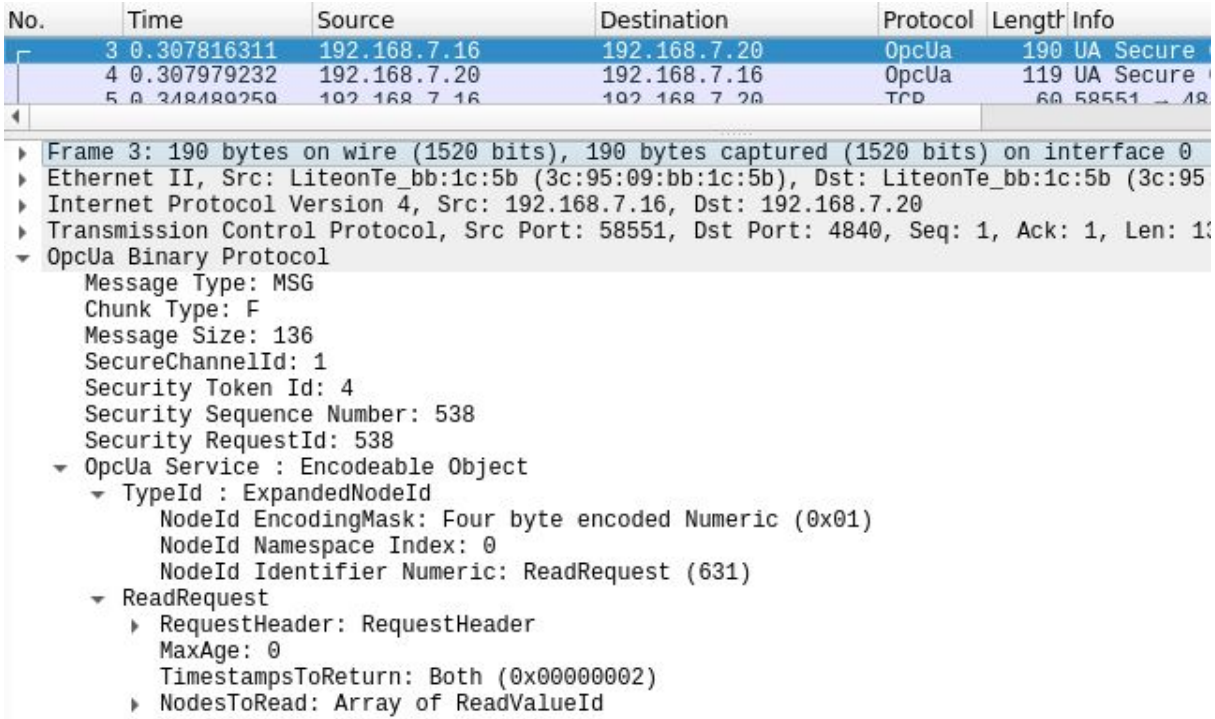
Resultados: Capturas ejemplos



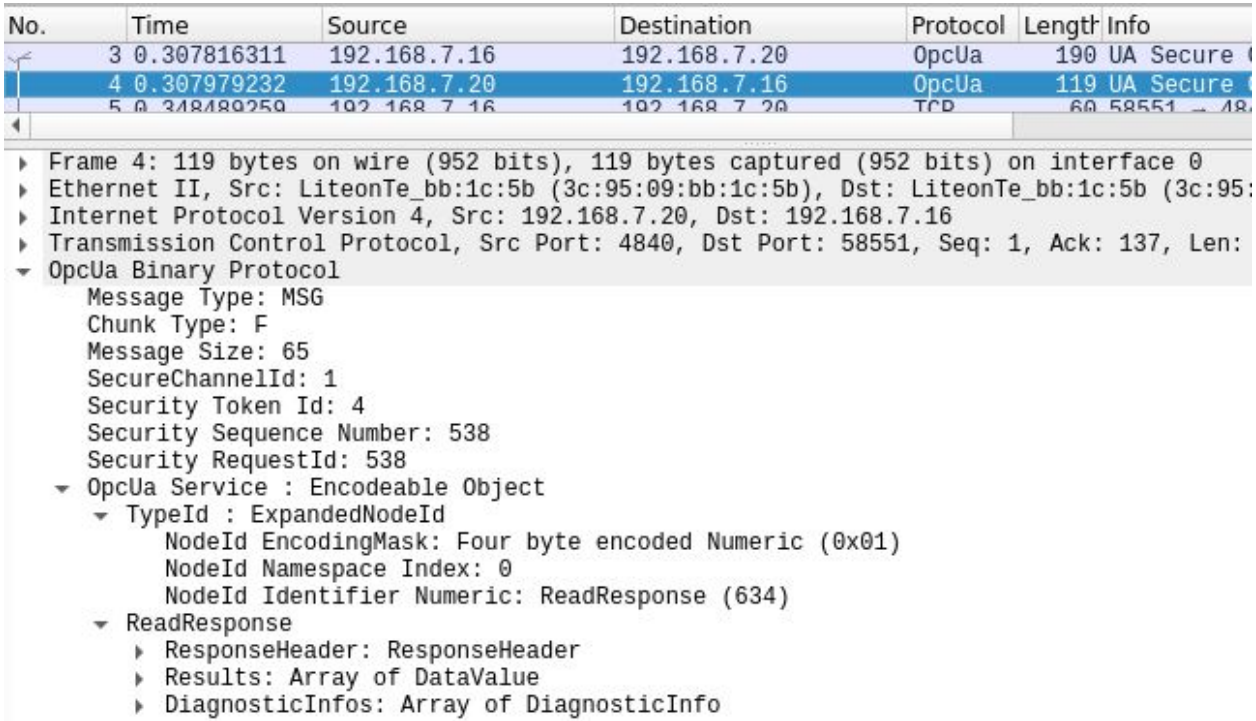
Source	Destination	Protocol	Length	Info
192.168.7.16	192.168.7.20	OpcUa	113	Hello message
192.168.7.20	192.168.7.16	TCP	54	4840 → 58848 [ACK] Seq=115252713
192.168.7.20	192.168.7.16	OpcUa	82	Acknowledge message

```
▸ Frame 199: 113 bytes on wire (904 bits), 113 bytes captured (904 bits) on interface 0
▸ Ethernet II, Src: LiteonTe_bb:1c:5b (3c:95:09:bb:1c:5b), Dst: LiteonTe_bb:1c:5b (3c:95
▸ Internet Protocol Version 4, Src: 192.168.7.16, Dst: 192.168.7.20
▸ Transmission Control Protocol, Src Port: 58848, Dst Port: 4840, Seq: 1, Ack: 115252713
▾ OpcUa Binary Protocol
    Message Type: HEL
    Chunk Type: F
    Message Size: 59
    Version: 0
    ReceiveBufferSize: 655360
    SendBufferSize: 655360
    MaxMessageSize: 0
    MaxChunkCount: 0
    EndPointUrl: opc.tcp://192.168.7.20:4840
```

Capturas


No. Time Source Destination Protocol Length Info
3 0.307816311 192.168.7.16 192.168.7.20 OpcUa 190 UA Secure
4 0.307979232 192.168.7.20 192.168.7.16 OpcUa 119 UA Secure
Frame 3: 190 bytes on wire (1520 bits), 190 bytes captured (1520 bits) on interface 0
Ethernet II, Src: LiteonTe_bb:1c:5b (3c:95:09:bb:1c:5b), Dst: LiteonTe_bb:1c:5b (3c:95
Internet Protocol Version 4, Src: 192.168.7.16, Dst: 192.168.7.20
Transmission Control Protocol, Src Port: 58551, Dst Port: 4840, Seq: 1, Ack: 1, Len:
OpcUa Binary Protocol
Message Type: MSG
Chunk Type: F
Message Size: 136
SecureChannelId: 1
Security Token Id: 4
Security Sequence Number: 538
Security RequestId: 538
OpcUa Service : Encodeable Object
TypeId : ExpandedNodeId
NodeId EncodingMask: Four byte encoded Numeric (0x01)
NodeId Namespace Index: 0
NodeId Identifier Numeric: ReadRequest (631)
ReadRequest
RequestHeader: RequestHeader
MaxAge: 0
TimestampsToReturn: Both (0x00000002)
NodesToRead: Array of ReadValueId

Capturas



No.	Time	Source	Destination	Protocol	Length	Info
3	0.307816311	192.168.7.16	192.168.7.20	OpcUa	190	UA Secure
4	0.307979232	192.168.7.20	192.168.7.16	OpcUa	119	UA Secure
5	0.348489259	192.168.7.16	192.168.7.20	TCP	60	58551 → 48

```
▸ Frame 4: 119 bytes on wire (952 bits), 119 bytes captured (952 bits) on interface 0
▸ Ethernet II, Src: LiteonTe_bb:1c:5b (3c:95:09:bb:1c:5b), Dst: LiteonTe_bb:1c:5b (3c:95:
▸ Internet Protocol Version 4, Src: 192.168.7.20, Dst: 192.168.7.16
▸ Transmission Control Protocol, Src Port: 4840, Dst Port: 58551, Seq: 1, Ack: 137, Len:
▾ OpcUa Binary Protocol
    Message Type: MSG
    Chunk Type: F
    Message Size: 65
    SecureChannelId: 1
    Security Token Id: 4
    Security Sequence Number: 538
    Security RequestId: 538
  ▾ OpcUa Service : Encodeable Object
    ▾ TypeId : ExpandedNodeId
        NodeId EncodingMask: Four byte encoded Numeric (0x01)
        NodeId Namespace Index: 0
        NodeId Identifier Numeric: ReadResponse (634)
    ▾ ReadResponse
      ▸ ResponseHeader: ResponseHeader
      ▸ Results: Array of DataValue
      ▸ DiagnosticInfos: Array of DiagnosticInfo
```